

From Chips to Chiplets: Architecture, Interconnects, and Challenges

Priyanka Singla

Baya Systems

priyanka.singla@bayasystems.com

Agenda

- Why Chiplets?
- What are Chiplets
- Challenges: The Fabric Problem
- Industry Example
- Research Questions
- Summary

Moore's Law & The End of Free Scaling

Year	Chip	Transistors
1971	Intel 4004	2,300
2000	Intel Pentium 4	42 Million
2020	Apple M1	16 Billion
2024	Apple M4 (3 nm)	~28 Billion
2024	NVIDIA GB200 GPU	~92 Billion

Moore's Law:

Transistor density doubles ~every 2 years

Dennard Scaling:

As transistors shrink, power density stays constant

→ **COLLAPSED ~2005 due to leakage currents**

What followed:

- Clock speeds plateaued at ~3–4 GHz
- Pivot to multicore & heterogeneous compute
- Process nodes continued, with diminishing returns

The Yield Problem

Murphy Yield Model:

$$Y = \frac{1 - e^{-AD^2}}{AD}$$

A = die area (cm²)

D = defect density (defects/cm²)

Node	D /cm ²	100mm ² Yield	400mm ² Yield
28 nm	~0.10	~90%	~67%
5 nm	~0.15	~86%	~55%
3 nm	~0.20	~82%	~45%

- A 400mm² monolithic SoC at 5 nm → ~55% yield
- Split into 4 × 100mm² chiplets → ~86% per die
- Defective chiplets discarded individually, not entire assembled systems

Key concept: Known Good Die (KGD)

Test each chiplet before assembly. Only assemble verified working dies.

Bigger nodes have higher yields, easier to manufacture, then what is the issue



Bigger != Better

The Yield Problem



We are hitting the **three** walls

Performance wall: $\propto f \cdot \text{IPC}$ (depends on transistor and interconnect delay, latter does not scale well; f caps)

Memory wall: Caches need to be physically close, but interconnect does not scale. Longer delays

Power wall: Dennard scaling is dead, cannot keep increasing frequency $\rightarrow$ thermal limits

Are we good,
we can have
as many as
advanced
nodes!!



Parameters	28nm Die	3nm Die
Yield	95%	82%
Performance	Baseline	Very high
Power	High	Much lower
Area	100mm ²	15-20mm ²

At 3nm, we can have multiple dies in 100mm²

Advanced nodes recover yield by going small!!

Advanced nodes are **Expensive!!**

Process Node	~Wafer Cost (USD)
28 nm	\$3,000
7 nm	\$9,000–10,000
5 nm	\$13,000
3 nm	\$15,000-19,000
2 nm	\$25,000+ (est.)

Monolithic SoCs

Low latency

Display, memory
channels controls

Yield penalty

Interconnect
(could be a xbar,
NoC, bus, ring)

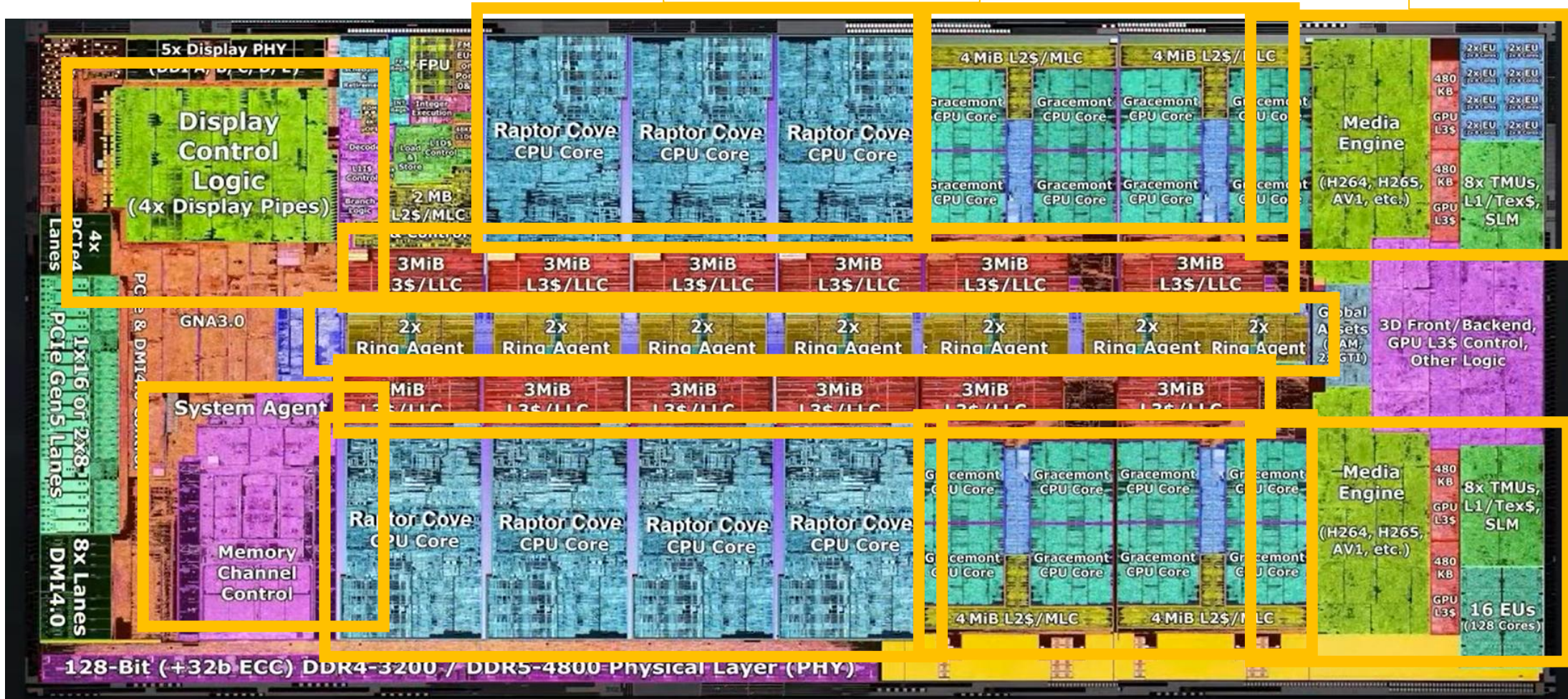
Same process node

Raptor Cove (P-core)

LLC slices

Gracemont cores (E)

GPUs,
TMUs, EUs



Intel core i9-13900K- Monolithic

Image src: [CHIPLETS: Divide and Conquer | The Future of Processors - YouTube](#)

Agenda

- Why Chiplets?
- What are Chiplets
- Challenges: The Fabric Problem
- Industry Example
- Research Questions
- Summary

Chiplets

A chiplet is a small, modular die designed to be combined with other dies to form a complete functional system, using a standardized die-to-die interface.

Fabricate only what NEEDS cutting-edge nodes (CPU cores, ML accelerators, GPU SMs) at the expensive advanced node.

Function	Node	Why
CPU cores	3–5 nm	Max performance w/ density
GPU SMs	3-5 nm	Throughput
ML accelerators	3-5 nm	Energy Efficiency
SRAM (cache)	5–7 nm	Bitcell stability floor
I/O PHYs, SerDes, Analog	28–65 nm	ESD, high-voltage, analog process
HBM / DRAM	DRAM node	Separate fab, stacked in package

Process specialization

**IP reuse
(AMD)**

Improved yield

2D ICs

Si Die (Transistors and metal wires)

Metal pad on die

Copper solder bumps (c4)

Cu pads on package substrate

Cu routing inside substrate

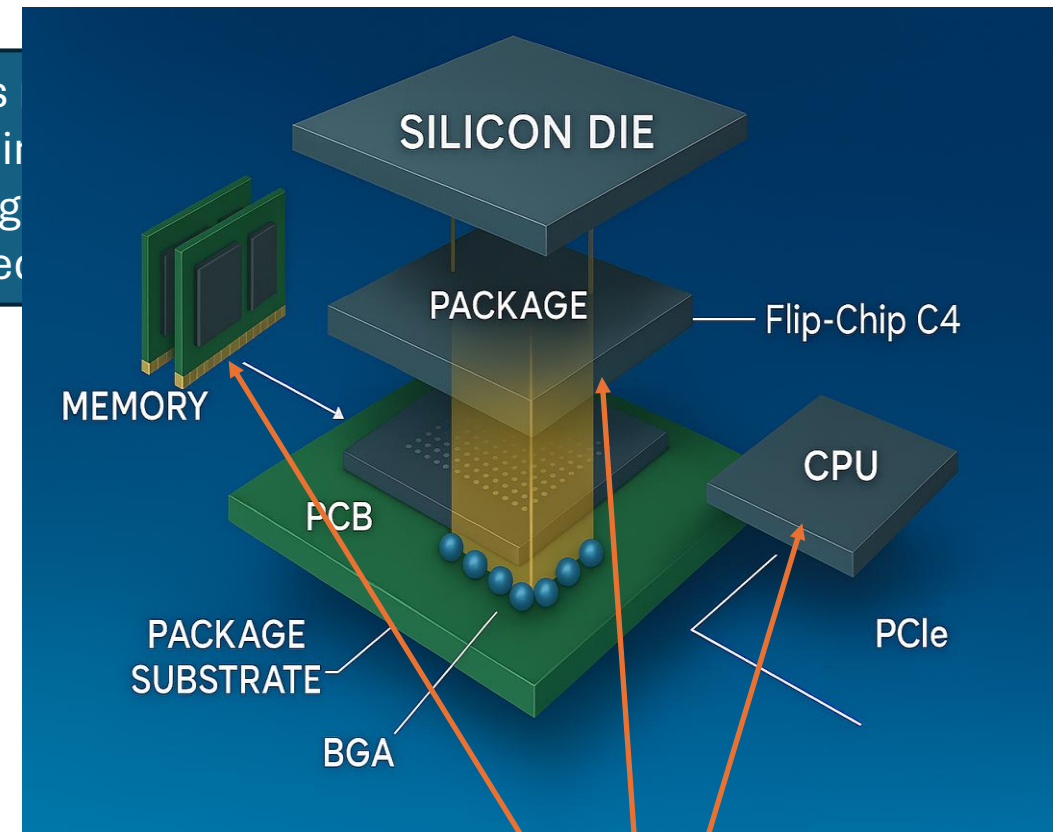
BGA solder balls

PCB copper trace

**10X denser
than BGA balls**

2D: Single die is substrate with its own I/O running through simple, but limited

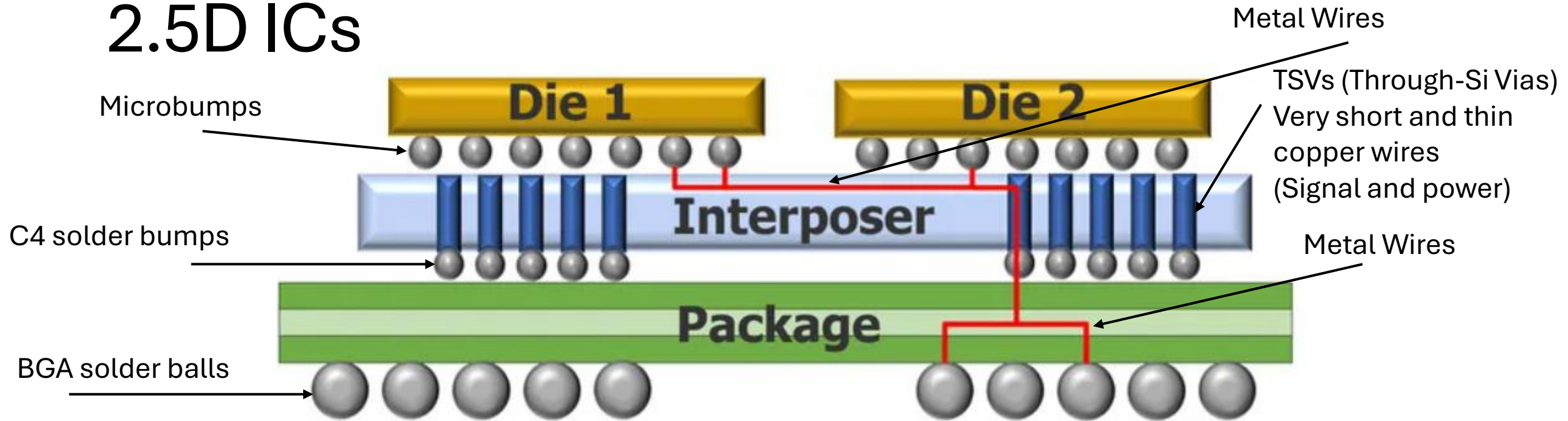
- **2D is cheap, manufacturable with existing PCB technology**
- **Can use if chiplets do not need to talk to each other frequently**



Different components connected metal wires (PCB trace) on the PCB
(longer latency)

Technology	Bump Pitch	I/O Density	Example
PCB BGA	~1 mm	~1 I/O/mm ²	Any board-level chip
Flip-chip C4	100–150 μm	40–100 I/O/mm ²	Standard SoC

2.5D ICs



Technology	Bump Pitch	I/O Density	Example
PCB BGA	~1 mm	~1 I/O/mm ²	Any board-level chip
Flip-chip C4	100–150 μm	40–100 I/O/mm ²	Standard SoC
2.5D micro-bump	40–55 μm	330–625 I/O/mm ²	TSMC CoWoS, Intel EMIB

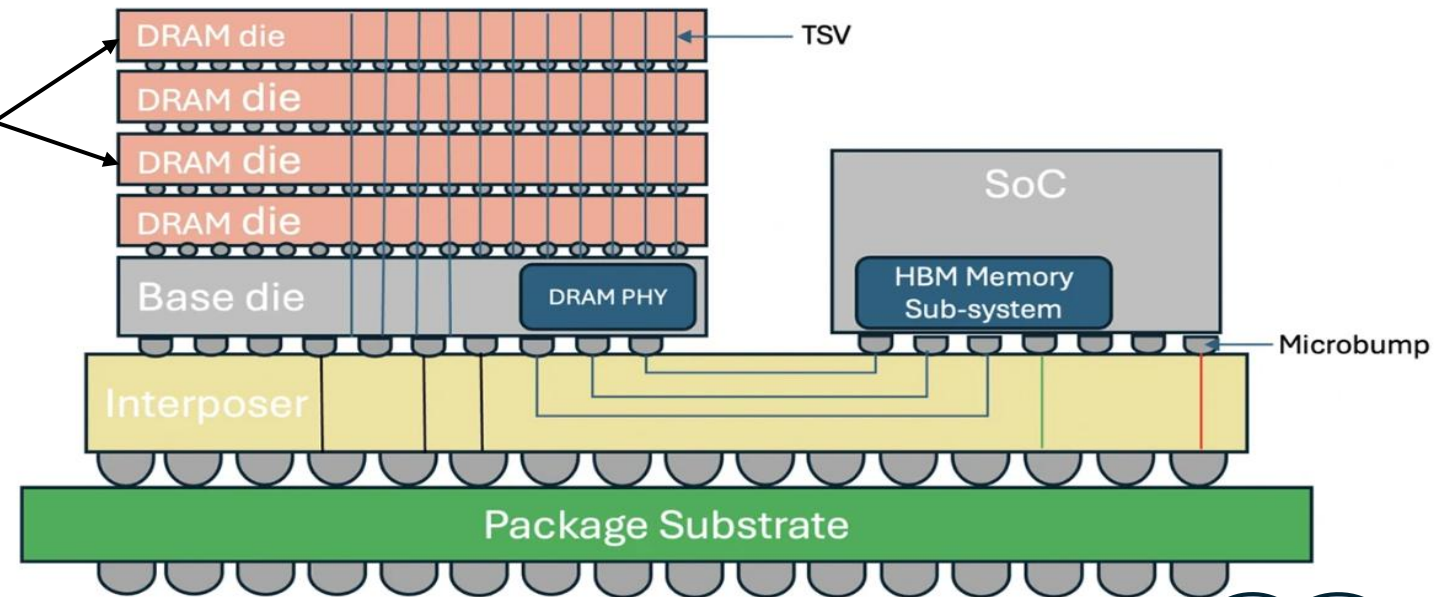
Higher b/w, Most common design

3D ICs

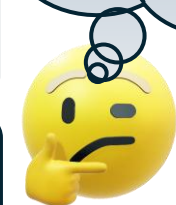
Aspect	TSV-based 3D	Hybrid-bond 3D
Signal connection	TSV	Direct cu-cu
Vertical pitch	~10um	~1-10um
Density	Medium	Extreme
TSV usage	Signals+power	Mostly power

Technology	Bump Pitch	I/O Density	Example
PCB BGA	~1 mm	~1 I/O/mm ²	Any board-level chip
Flip-chip C4	100–150 μm	40–100 I/O/mm ²	Standard SoC
2.5D micro-bump	40–55 μm	330–625 I/O/mm ²	TSMC CoWoS, Intel EMIB

Stacked Dies



This is perfect, we should always use 3D ICs?



Thermal issues – heat must go through layers – can have limited height

Yield sensitive – one bad layer can kill the stack

Expensive, complex to repair

2D vs 2.5D vs 3D

Aspect	2D	2.5D	3D
Dies	One/Many on PCB	Many in one package	Stacked
Die-to-Die wiring	PCB	Interposer	Direct
TSV role	None	Power / I/O	Mostly power
Bandwidth	Low	High	Very high
Bump Type	C4 + BGA	Micro Bump+C4	Hybrid Bond
Energy per bit	High	Low	lowest
Thermal	Easy	Moderate	Hard

Chiplet-based Systems in Industry

Company	Product	Chiplet Structure
AMD	EPYC Genoa	12 CPU chiplets (5 nm) + 1 I/O die (6 nm) — up to 96 cores
AMD	Instinct MI300X	13 chiplets: GPU + CPU + HBM — first heterogeneous AI package
Intel	Meteor Lake	CPU + GPU + I/O + SoC tiles connected via EMIB bridges
NVIDIA	Grace Hopper GH200	CPU + GPU linked via NVLink-C2C at 900 GB/s bidirectional
Apple	M1 Ultra	Two M1 Max dies connected via UltraFusion bridge
Marvell	OCTEON 10 DPU	Multi-tile: compute + I/O + security chiplets

Agenda

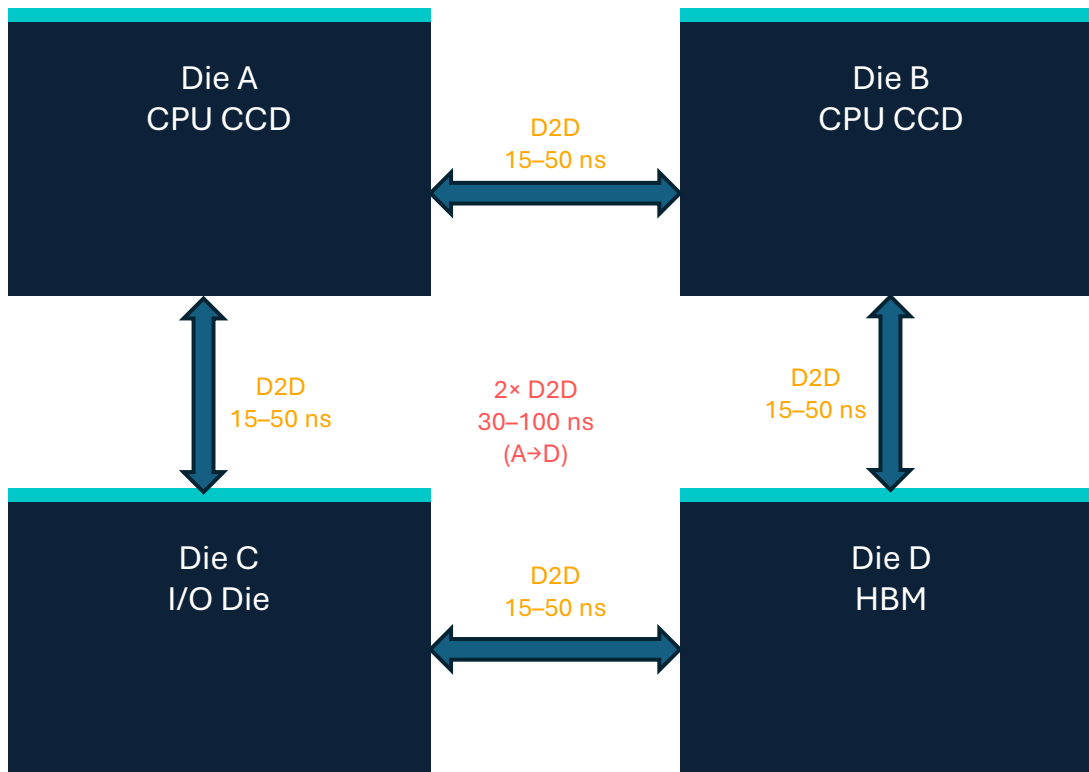
- Why Chiplets?
- What are Chiplets
- Challenges: The Fabric Problem
- Industry Example
- Research Questions
- Summary

The Die Boundary Is Not Just a Wire

Metric	On-Die Wire	D2D Link (UCIe)	Off-Package (PCB)
Energy/bit	0.01-0.1 pJ	0.5-3 pJ	5-20 pJ
Latency	0.1-2 ns	15-50 ns	50-500 ns
Bandwidth	TB/s (wire count scales freely)	Capped by bump count	Limited by PCB trace count
Wire density	Millions / mm ²	~1,000 bumps / mm ²	Tens / cm
Coherence	Native, cheap	Possible, costs D2D RTT	Avoided entirely

Crossing a die boundary costs 30–100× more energy and 10–30× more latency than an on-die wire.

Topology: Package NUMA and the Routing Challenges



Access Latency Asymmetry: Latency to access data depends on WHICH die holds it.

Local (within same die): 5 – 15ns

Die A → Die B: 1 D2D crossing (~15–50 ns)

Die A → Die D: 2 D2D crossings (~30–100 ns)

Traditional Nocs had uniform hop cost. In chiplets the hop-cost varies by 2-10x.

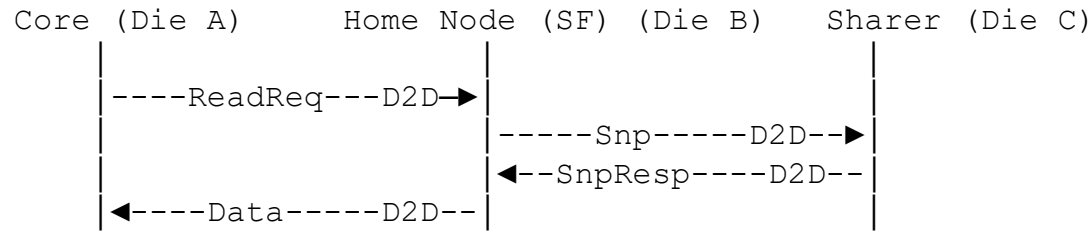
Routing algorithm challenges:

- Standard dimension-order routing fails here; assumes uniform hop cost
 - Load balancing must account for link costs, not just hops
 - Adaptive routing needs congestion feedback across dies — but feedback is stale (1 RTT)
 - Routing oscillation: stale congestion info causes traffic to swing between paths
- HN-F placement determine average snoop distance

The fabric must be topology-aware and NUMA-aware by design.

Cache Coherence: Every Snoop Costs a D2D Round Trip

CHI read miss – cross-die worst case:



Total: 4 × D2D latency
= 60-200 ns per coherence transaction
vs. 5-20 ns for intra-die LLC access (monolithic dies)

Home Node (HN) placement is critical:

Option A — One global HN on one die

✓ Simple directory.

✗ All traffic crosses D2D

Option B — HN distributed across CPU dies – one HN per chiplet cluster:

✓ Local snoops stay local

✗ Larger distributed directory

AMD's answer (EPYC):

Centralised HN on dedicated I/O die.

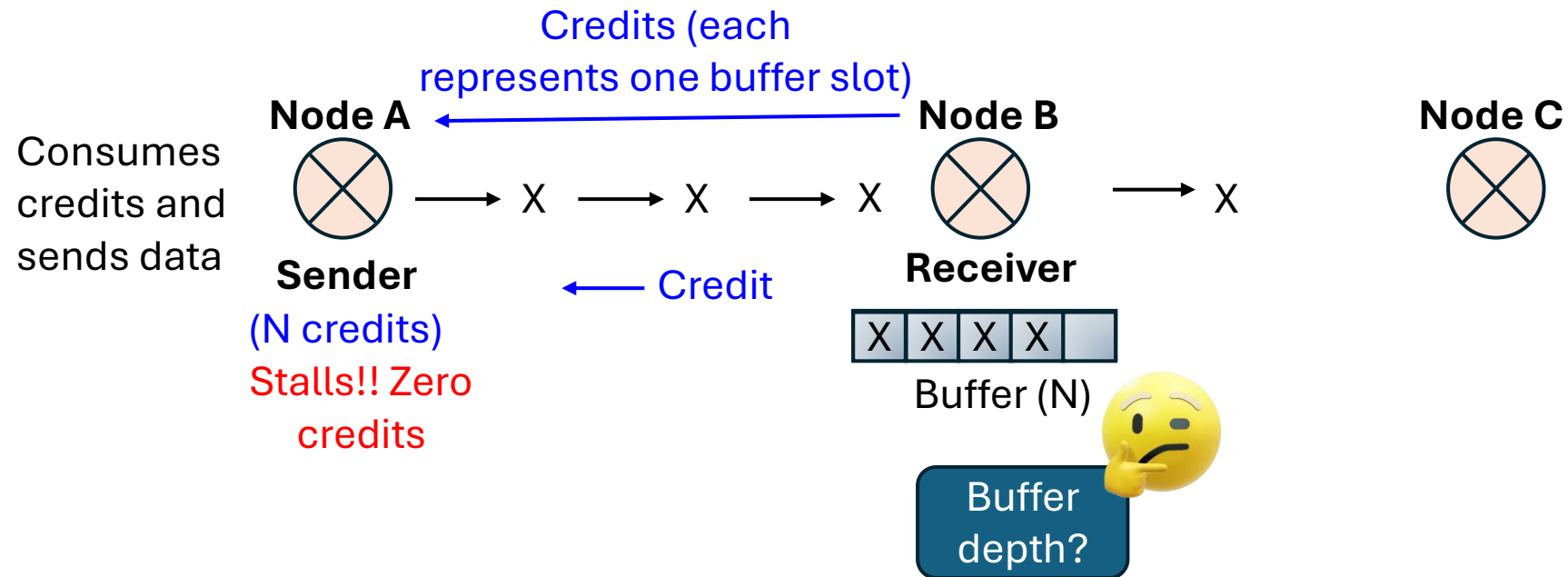
All CPU CCDs are pure requestors (RN only).

Single consistent ownership/ordering point.

The snoop filter must be sized for the cross-die working set
under-sized SF → spurious snoops → wasted D2D bandwidth.

Baya supports UMA / NUMA / sub-NUMA (bayasystems.com).

Flow Control

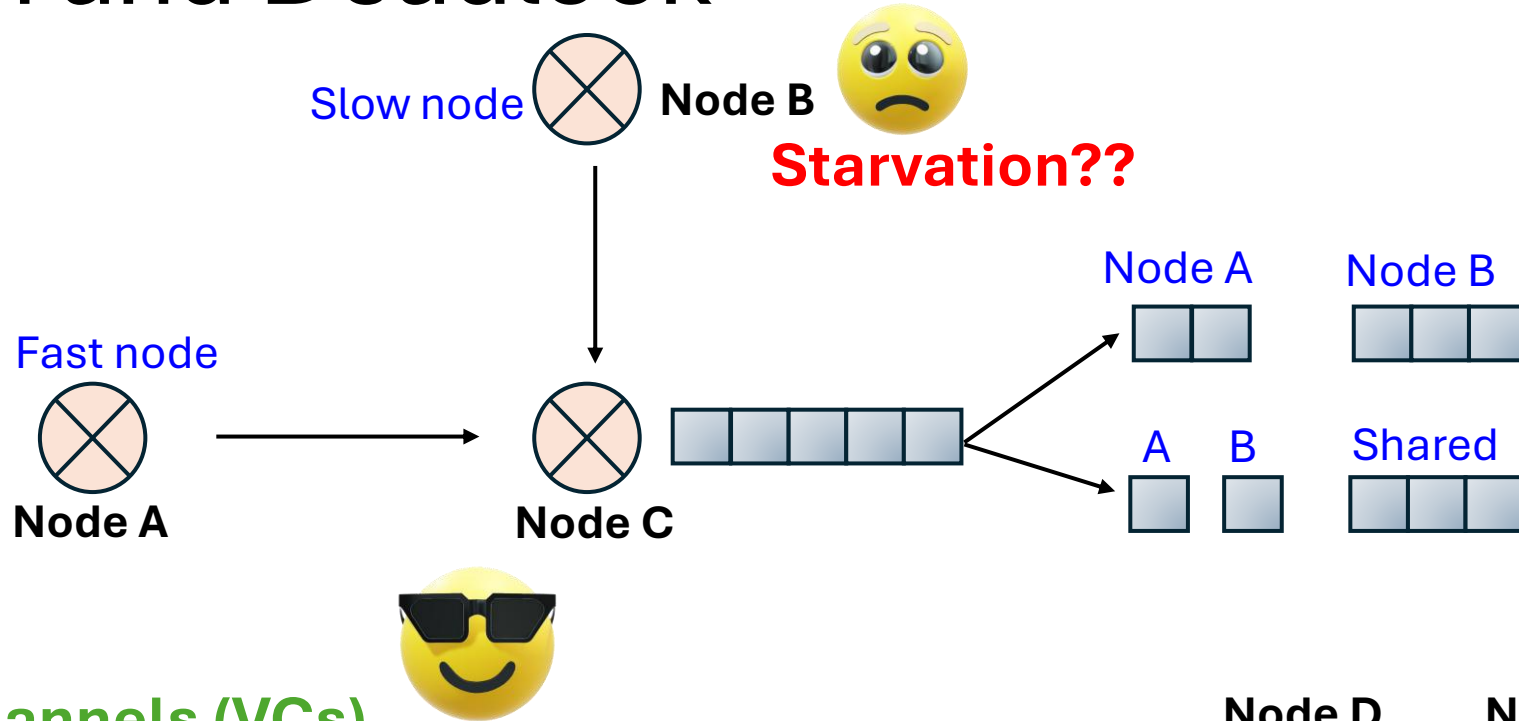


$$\text{BDP} = \text{BW} \times \text{RTT}_{\text{link}}$$

On-die (5 ns RTT, 100 GB/s): 500 B

D2D UCle (40 ns, 4 TB/s): 160 KB

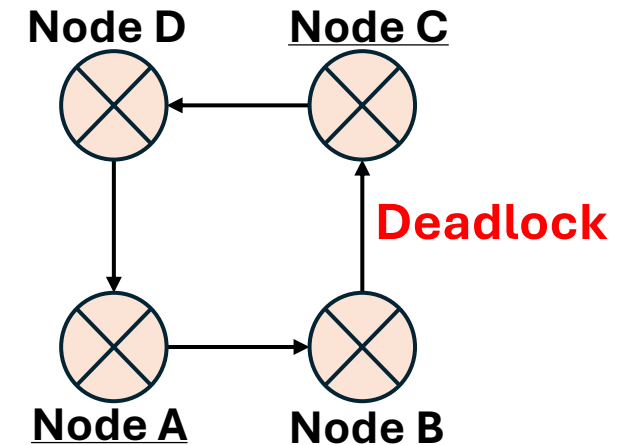
Starvation and Deadlock



Virtual Channels (VCs)

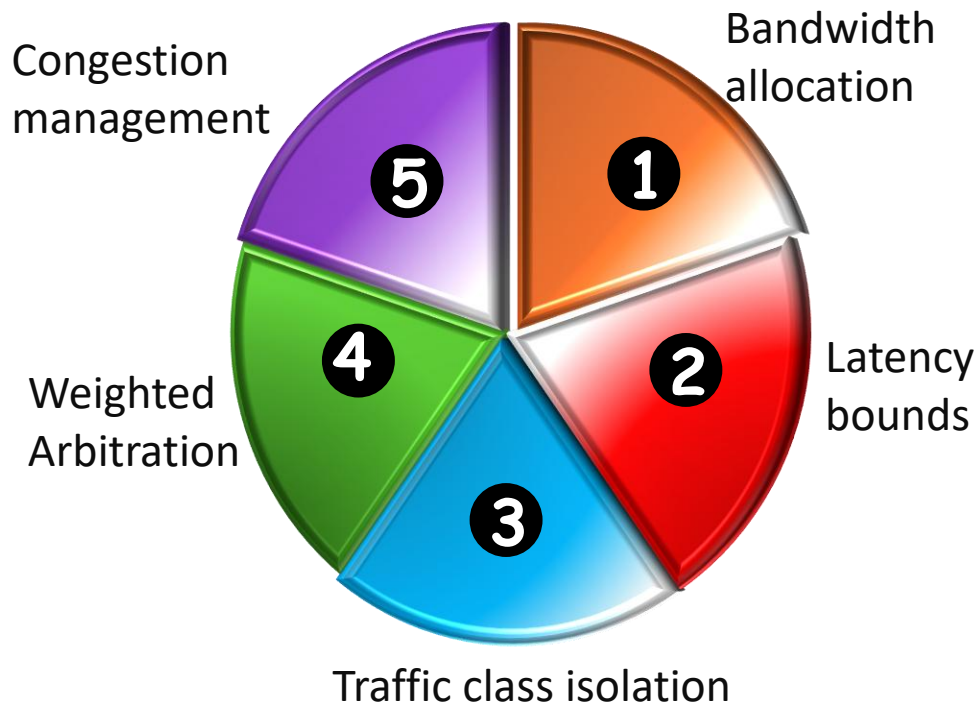
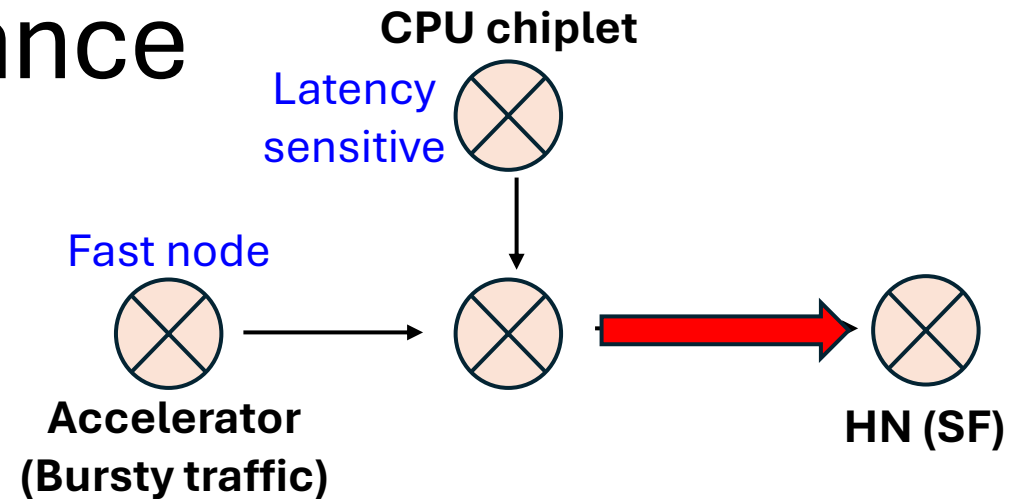
One **physical link** shared by N independent logical channels.
Each VC has its own credit counter and buffer.

Turn model / VC DAG.



QoS: Guaranteeing Performance

- A D2D link is a shared, bandwidth-constrained resource.
- A bursty AI accelerator on one chiplet can saturate it and starve latency-sensitive CPU coherence traffic on another.
- Without explicit QoS enforcement, there are no performance guarantees.



Why QoS Is Harder in Chiplet Systems:

- D2D link BW is physically capped by bump count -- over-subscription is not recoverable by adding wires (since by different vendors)
- QoS policy must be enforced across die boundaries; a router on chiplet B must respect priorities set on chiplet A
- NUMA asymmetry means the same traffic class has different latency budgets depending on which dies are involved
- Multiple protocols (CHI coherence + AXI streaming + AI collective) share the same physical D2D link

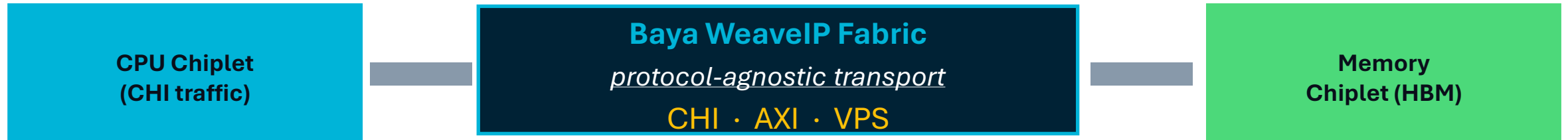
Agenda

- Why Chiplets?
- What are Chiplets
- Challenges: The Fabric Problem
- Industry Example
- Research Questions
- Summary

Industry Example: Baya Systems



Builds the interconnect IP that links chiplets inside a package -- the fabric that makes a chiplet system behave like one coherent chip



Connecting the dots: From Problems to Baya Systems Solutions

WeaveIP

Scale-up interconnect
within one package
4 TB/s · Multiple VCs · 3 GHz · AMBA 5

NeuraScale

Scale-out fabric
across packages
32 PB/s bisection BW · AI training clusters

WeaverPro

Verification & QoS
toolchain
*Formal deadlock proof · KGD test · RAS
built-in*



Connecting the Dots: From Problem to Baya's Solution

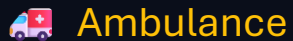
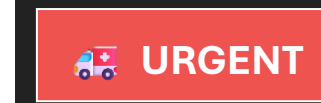
Challenge	Traditional NoC fails because...	Baya's Design Choice	Result
Multiple protocols (CHI, AXI, AI) on one shared D2D link	Build a separate NoC for each protocol. Adding CXL = redesign the entire router. Triples silicon area.	Protocol-agnostic transport layer. adapters at chiplet boundary handle each protocol.	Same fabric IP reused for CHI + CXL + AI. No router redesign when new protocol is added.
critical CPU coherence snoops	No traffic priority. CPU snoop queues behind GB-scale AI transfer ? 500+ ns.	Strict priority for coherence traffic. Weighted arbitration for bulk.	200 ns even under full AI load.
Mesh NoC wastes area and scales poorly beyond 16 tiles	Area grows $O(N^2)$ with tile count. Long hop count raises latency. Bisection bandwidth bottlenecks.	Hierarchical topology with verifiable die boundary (cluster perimeter). Physically- aware tiling.	2× smaller than mesh. 4 TB/s single cluster. 32 PB/s bisection. BW at system scale.
Hard to verify correctness across die boundaries	Ad-hoc integration testing post assembly. Defect found after bonding wastes the entire package.	Correct-by-construction RTL. Per-cluster KGD testability before assembly. Formal deadlock proof.	Ships as hardened IP. 3 GHz · AMBA 5. RAS + QoS built-in. Test each chiplet before bonding.

Baya Systems builds the interconnect fabric that links chiplets inside a package. Each design choice below directly addresses a failure mode of traditional NoC approaches.

The Key Idea: Protocol vs Transport

A chiplet package carries CPU traffic, AI traffic, and memory traffic — all at once, on shared wires. How do you build one fabric that handles all of them?

The Analogy



Ambulance

CPU memory request (urgent, blocking)



Delivery truck

AI bulk data transfer (large, not urgent)

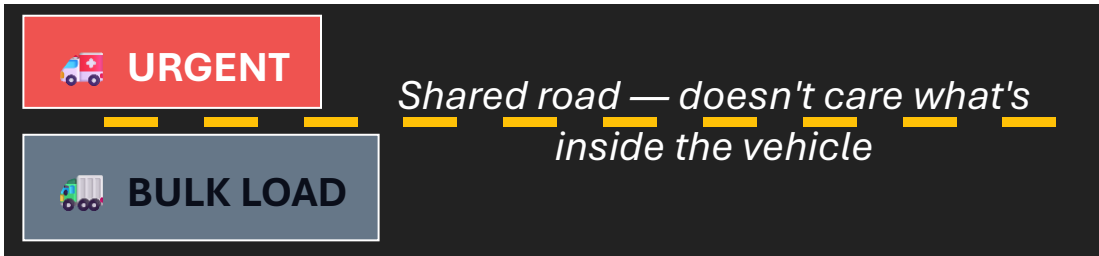
New vehicle type = New road



The Key Idea: Protocol vs Transport

A chiplet package carries CPU traffic, AI traffic, and memory traffic — all at once, on shared wires. How do you build one fabric that handles all of them?

The Analogy



The road asks only two questions:

Where are you going? Are you urgent?

Baya's way: One road. Every vehicle gets a label at the entrance (destination + urgency).

Road just moves it. Label removed at exit.

On a Chip (Baya's Fabric)

Road network	Baya transport fabric
Entrance label	Adapter: adds {destination + priority}
Exit label removal	Adapter: reconstructs original format

How it flows:



Fabric sees only: {destination, priority}. Contents are opaque.

New vehicle type = new entrance adapter only. Road unchanged.

Add a new protocol

Just write a new adapter. The fabric routers don't change.

Mix CHI + AXI + AI traffic

All use the same wires. Adapters translate at each end.

Apply QoS across all traffic

One priority mechanism (VCs) works for every protocol.



Scale-Up vs Scale-Out: Ways to Build Larger Systems

SCALE-UP -- more chiplets in one package

- All chiplets sit on the same silicon interposer. Signals travel through dense metal wires at microbump pitch, never leaving the package.
- Note: chiplets can still use different protocols (e.g. CHI for CPU, AXI for AI accelerator) -- protocol adapters handle translation at each chiplet boundary.
- What scale-up gives you is latency low enough that coherence across dies is practical.

Latency
5 – 50 ns

Energy
0.5 – 3 pJ/bit

Coherence
Native (free)

Hard limit:

Package size, bump count, and thermal budget cap you at ~8–16 chiplets. Beyond that, you must scale out.

Use scale-up when:

Tightly coupled compute needs low latency + coherence (CPU + HBM + AI in one package)

SCALE-OUT -- multiple packages connected

No physical ceiling

Cannot fit 1000 AI chips in one package. Scale-out has no upper limit.

Fault isolation

One package fails → rest keep running. One bad bump ≠ full system down.

Composability

Mix CPU + GPU + memory packages in any ratio. Reconfigure per workload.

AI training workloads

LLM training (tensor/pipeline/data parallelism) tolerates 100–1000 ns latency by design.

Use scale-out when:

Loosely coupled bulk exchange, no coherence needed (LLM training across 1000 GPU packages)

Traditional NoC vs Chiplet Fabric: What Changes

Dimension	Traditional On-Die NoC	Chiplet Fabric
Link cost	ps latency · fJ/bit · add wires freely	ns latency · pJ/bit · bandwidth $\propto$ bump count
Topology	Symmetric mesh/ring · uniform hop cost	Asymmetric NUMA · per-link BW/latency modelled
Protocol coupling	Fabric is CHI-native or AXI-native · bridges add latency	Transport separated from protocol · CHI/AXI/CXL = adapters
Coherence	Flat single-die directory · snoops $\approx$ 5 ns	Hierarchical UMA/NUMA/sub-NUMA · local HN-F per cluster
Die boundary	Does not exist · no perimeter · no KGD interface	First-class object · verifiable perimeter · congestion isolation
Flow control	Credit pool sized for on-die RTT · few KB buffer	Credit pool sized for D2D RTT · 160 KB at 4 TB/s
Physical design	Router placement free · P&R tool decides	Tiling constrained by chiplet floorplan + bump-map
Testing	Full-SoC test only · no independent unit test	KGD-testable per cluster · formal perimeter specification

Agenda

- Why Chiplets?
- What are Chiplets
- Challenges: The Fabric Problem
- Industry Example
- Research Questions
- Summary

Research Problems

Composable Fabric Performance Model

Can a single analytical model bound latency/BW for concurrent coherent, streaming, and AI collective traffic on the same fabric?

Home Node Placement in a Multi-Die System

What is the optimal HN-F placement policy for a given chiplet topology and workload mix? When does replication help vs. hurt? How many are needed.

Adaptive Routing Stability

Adaptive routing improves utilization but causes oscillation under NUMA load. What routing algorithm properties guarantee stability and livelock freedom?

Multi-Protocol Deadlock Freedom

Verifying VC DAG acyclicity is polynomial. Full protocol-level deadlock freedom under mixed CHI/CXL/AXI is undecidable in general. What are tractable sufficient conditions?

Agenda

- Why Chiplets?
- What are Chiplets
- Challenges: The Fabric Problem
- Industry Example
- Research Questions
- Summary

Summary

Why Chiplets	Monolithic scaling hit the wall at 3–5 nm. Yield advantage + composability + IP reuse + cost effectiveness. The die boundary is now a first-class design constraint.
The Fabric Problem	D2D links latency and energy costs higher than on-die wires. NUMA asymmetry, protocol heterogeneity, and bump-count constraints make chiplet NoC a fundamentally different problem class.
Coherence Cost	Every cross-die snoop costs 2 D2D round trips (60–200 ns). Home node placement and snoop filter sizing are architectural decisions with large performance impact.
QoS & Deadlock	Multiple VCs per network isolate CHI channels + different classes. VC dependency graph must be a DAG, proven formally, not just tested.
Open Research	Composable performance model · hierarchical coherence scaling · home node placement · adaptive routing scaling.

"The future bottleneck is not compute — it is communication."